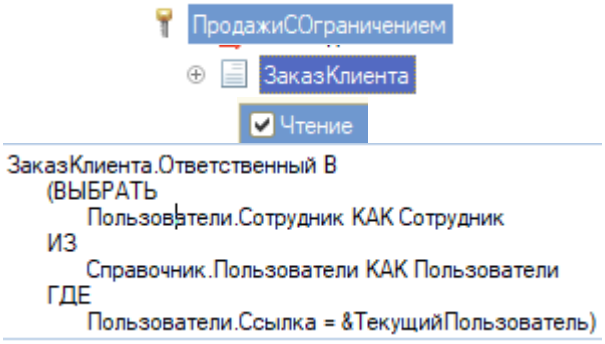
Ограничение доступа к заказам

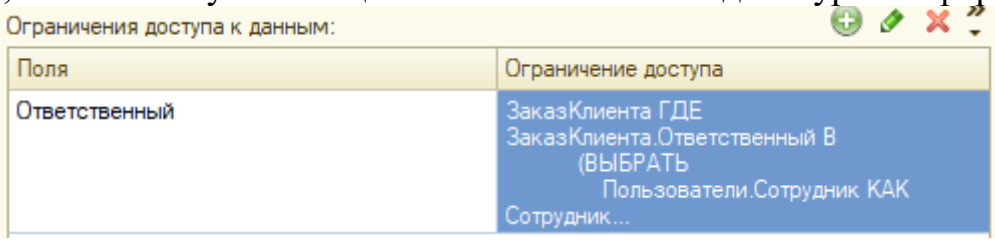
Нужно создать роль «Продажи (ограниченно)». Набор прав должен быть идентичен набору прав роли «Продажи».
Нужно ограничить доступ к таблице документа «Заказ клиента»: пользователь с ролью «Продажи (ограниченно)» должен видеть только заказы, в которых он указан как ответственный.

Для этого создадим параметр сеанса ТекущийПользователь, который будет ссылаться на справочник пользователи

В таблице документа, показывает заказы, для конкретной роли, которая указана, как ответственная

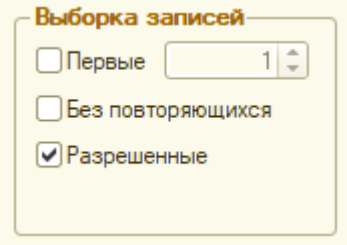


Так же мы можем наложить ограничения на конкретные поля, чтобы не прятать всю строку, и в итоге получится так, что мы ограничим только доступ к полю, в котором ответственный не тот, что указан в параметре сеанса. В случае, если мы, допустим, уберем видимость поля, но для ограничения все равно его нужно учитывать, то воспользуемся опцией Использовать всегда на уровне формы



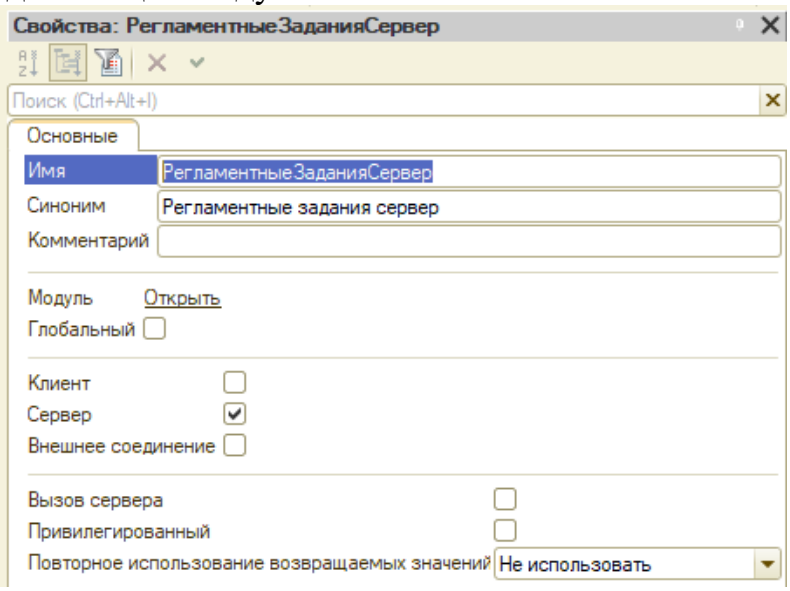
Представим ситуацию, что мы получаем список документов и ответственных за эти документы в запросе. У нас получается ситуация, что выборка содержит

данные, которые не доступны пользователю и выдает ошибку, которую можно обойти с помощью:



Регламентные задания:

Для начала создадим общий модуль:

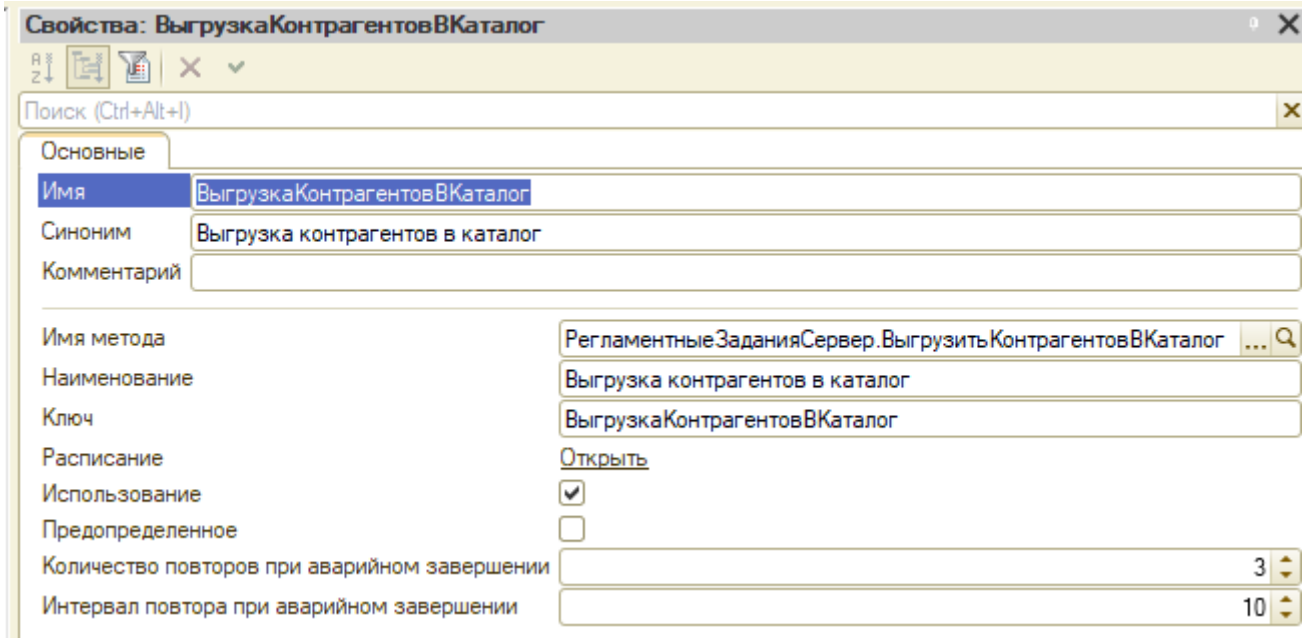


Процедура внутри модуля обязательно должна быть экспортная

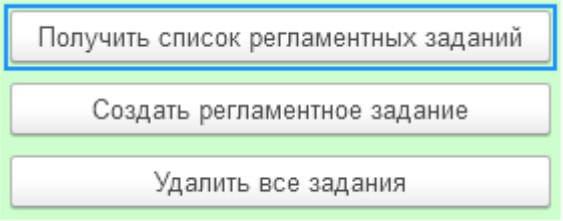
Допустим, что модуль у нас содержит следующий код:

```
Процедура ВыгрузитьКонтрагентовВКаталог() Экспорт

    Запрос = Новый Запрос;
    Запрос.Текст =
        "ВЫБРАТЬ
        |   Контрагенты.ИНН КАК УНП,
        |   Контрагенты.Наименование КАК Наименование,
        |   Контрагенты.НаименованиеПолное КАК НаименованиеПолное
        |ИЗ
        |   Справочник.Контрагенты КАК Контрагенты
        |ГДЕ
        |   НЕ Контрагенты.ПометкаУдаления";

    РезультатЗапроса = Запрос.Выполнить();

    Выборка = РезультатЗапроса.Выбрать();

    Текст = Новый ТекстовыйДокумент;
    Пока Выборка.Следующий() Цикл
        ТекСтрока = Выборка.УНП + ";" + Выборка.Наименование + ";" + Выборка.НаименованиеПолное;
        Текст.ДобавитьСтроку(ТекСтрока);
    КонецЦикла;

    Текст.Записать("E:\temp\СписокКонтрагентов.txt", КодировкаТекста.UTF8);

КонецПроцедуры
```

Создадим регламентное задание:



Реализация основных команд при работе с регламентными заданиями



1. Получить список регламентных заданий

```
&НаСервере
Процедура ПолучитьСписокРегламентныхЗаданийНаСервере()

    МассивЗаданий = РегламентныеЗадания.ПолучитьРегламентныеЗадания();

    Для каждого Задание Из МассивЗаданий Цикл
        НоваяСтрока = Объект.СписокРегламентныхЗаданий.Добавить();
        ЗаполнитьЗначенияСвойств(НоваяСтрока, Задание);
    КонецЦикла;

КонецПроцедуры

&НаКлиенте
Процедура ПолучитьСписокРегламентныхЗаданий(Команда)
    ПолучитьСписокРегламентныхЗаданийНаСервере();
КонецПроцедуры
```

1. Создание регламентного задания

```
&НаКлиенте
Процедура СоздатьРегламентноеЗадание(Команда)
    СоздатьРегламентноеЗаданиеНаСервере();
КонецПроцедуры

&НаСервере
Процедура СоздатьРегламентноеЗаданиеНаСервере()

    НовоеЗадание = РегламентныеЗадания.СоздатьРегламентноеЗадание(Метаданные.РегламентныеЗадания.ВыгрузкаКонтрагентовВКаталог);
    НовоеЗадание.Использование = Истина;

    Расписание = Новый РасписаниеРегламентногоЗадания;
    Расписание.ПериодПовтораДней = 1;//каждый день
    Расписание.ПериодПовтораВТечениеДня = 60;//каждые 60 секунд

    НовоеЗадание.ИмяПользователя = "Администратор";
    НовоеЗадание.Расписание = Расписание;

    НовоеЗадание.Записать();

КонецПроцедуры
```

1. Удалить

```
&НаКлиенте
Процедура УдалитьВсеЗадания(Команда)
    УдалитьВсеЗаданияНаСервере();
КонецПроцедуры

&НаСервере
Процедура УдалитьВсеЗаданияНаСервере()

    МассивЗаданий = РегламентныеЗадания.ПолучитьРегламентныеЗадания();
    Для каждого Задание Из МассивЗаданий Цикл
        Задание.Удалить();
    КонецЦикла;

КонецПроцедуры
```

Задача.
Загрузка курсов валют

Нужно реализовать функционал по ежедневной автоматической загрузке курсов валют с сайта Национального банка Республики Беларусь: http://www.nbrb.by/

```
Процедура ЗагрузкаКурсовВалютРБ() Экспорт

    ИмяСобытияЖурнала = "IRONSKILLS_ЗагрузкаКурсовВалют";

    ЗаписьЖурналаРегистрации(ИмяСобытияЖурнала, УровеньЖурналаРегистрации.Информация, , , "старт: " + ТекущаяДата());

    ДатаКурса = НачалоДня(ТекущаяДата());

    Запрос = Новый Запрос;
    Запрос.Текст =
        "ВЫБРАТЬ
        |   Валюты.СимвольныйКод КАК СимвольныйКод,
        |   Валюты.Ссылка КАК Ссылка
        |ИЗ
        |   Справочник.Валюты КАК Валюты
        |ГДЕ
        |   Валюты.ПометкаУдаления = ЛОЖЬ";

    РезультатЗапроса = Запрос.Выполнить();

    СоответствиеВалют = Новый Соответствие;
    Выборка = РезультатЗапроса.Выбрать();

    Пока Выборка.Следующий() Цикл
        СоответствиеВалют.Вставить(Выборка.СимвольныйКод, Выборка.Ссылка);
    КонецЦикла;

    HTTPСоединение = Новый HTTPСоединение("www.nbrb.by", ,,,, 20, Новый ЗащищенноеСоединениеOpenSSL);

    HTTPЗапрос = Новый HTTPЗапрос("/api/exrates/rates?ondate=" + Формат(ДатаКурса, "Д=yyyy-MM-dd") + "&periodicity=0");

    Попытка
        HTTPОтвет = HTTPСоединение.Получить(HTTPЗапрос);
    Исключение
        ТекстОшибки = "Не удалось выполнить HTTP-запрос, адрес ресурса: " + "www.nbrb.by" + "/api/exrates/rates?ondate=" + Формат(ДатаКурса, "Д=yyyy-MM-dd") + "&periodicity=0";
        ЗаписьЖурналаРегистрации(ИмяСобытияЖурнала, УровеньЖурналаРегистрации.Ошибка, , , ТекстОшибки);
    КонецПопытки;

    Если HTTPОтвет.КодСостояния = 200 Тогда

        ТелоОтвета = HTTPОтвет.ПолучитьТелоКакСтроку();

        ЧтениеJSON = Новый ЧтениеJSON;
        ЧтениеJSON.УстановитьСтроку(ТелоОтвета);

        МассивКурсов = ПрочитатьJSON(ЧтениеJSON);

        Для каждого СтруктураКурса Из МассивКурсов Цикл

            ВалютаССылка = СоответствиеВалют.Получить(СтруктураКурса.Cur_Abbreviation);

            Если ВалютаССылка <> Неопределено Тогда
                НоваяЗапись = РегистрыСведений.КурсыВалют.СоздатьМенеджерЗаписи();
                    НоваяЗапись.Период      = ДатаКурса;
                    НоваяЗапись.Валюта      = ВалютаССылка;
                    НоваяЗапись.Курс        = СтруктураКурса.Cur_OfficialRate;
                    НоваяЗапись.Кратность   = СтруктураКурса.Cur_Scale;
                    НоваяЗапись.Записать();
                Иначе

                КонецЕсли;

            КонецЦикла;

        Иначе

            ОтветСервера = HTTPОтвет.ПолучитьТелоКакСтроку();
            ТекстОшибки = "Ошибка выполнения запроса: код состояния "  + HTTPОтвет.КодСостояния + ", тело ответа: " + ОтветСервера;
            ЗаписьЖурналаРегистрации(ИмяСобытияЖурнала, УровеньЖурналаРегистрации.Ошибка,,,ТекстОшибки);

        КонецЕсли;

        ЗаписьЖурналаРегистрации(ИмяСобытияЖурнала, УровеньЖурналаРегистрации.Информация, , , "финиш: " + ТекущаяДата());

КонецПроцедуры
```

Работа с FTP

```
&НаКлиенте
Процедура ПутьКФайлуНачалоВыбора(Элемент, ДанныеВыбора, СтандартнаяОбработка)

    Проводник = Новый ДиалогВыбораФайла(РежимДиалогаВыбораФайла.Открытие);

    ЧтоДелатьПосле = Новый ОписаниеОповещения("ПослеВыбораФайла", ЭтотОбъект);
    Проводник.Показать(ЧтоДелатьПосле);

КонецПроцедуры
```

```
&НаКлиенте
Процедура ПослеВыбораФайла(ВыбранныеФайлы, ДополнительныеПараметры) Экспорт

    Если ВыбранныеФайлы = Неопределено Тогда
        Возврат;
    КонецЕсли;

    Объект.ПутьКФайлу = ВыбранныеФайлы[0];

КонецПроцедуры // ПослеВыбораФайла()
```

```
&НаКлиенте
Процедура ЗагрузитьФайлНаFTP(Команда)

    FTPСоединение = Новый FTPСоединение("195.69.187.77", , "user26101", "qNHzsRYBrF32", , Истина, 20)

    Попытка

        FTPСоединение.УстановитьТекущийКаталог("/");
        FTPСоединение.Записать(Объект.ПутьКФайлу, "/test_file.txt");

        Сообщение = Новый СообщениеПользователю;
        Сообщение.Текст = "Файл успешно загружен на FTP-сервер";
        Сообщение.Сообщить();

    Исключение

        Сообщение = Новый СообщениеПользователю;
        Сообщение.Текст = "Не удалось загрузить файл на FTP-сервер, описание ошибки: " + ОписаниеОшиб
        Сообщение.Сообщить();

    КонецПопытки;

КонецПроцедуры
```

```
&НаКлиенте
Процедура ПолучитьФайлСFTP(Команда)

    FTPСоединение = Новый FTPСоединение("195.69.187.77", , "user26101", "qNHzsRYBrF32", ,

    Попытка

        НайденныеФайлы = FTPСоединение.НайтиФайлы("/test_file.txt");
        Если НайденныеФайлы.Количество() <> 0 Тогда

            ДанныеФайлаFTP = НайденныеФайлы[0];

            FTPСоединение.Получить(ДанныеФайлаFTP.ПолноеИмя, Объект.ПутьКФайлу);

            Сообщение = Новый СообщениеПользователю;
            Сообщение.Текст = "Файл успешно получен с FTP-сервера";
            Сообщение.Сообщить();

        КонецЕсли;

    Исключение

        Сообщение = Новый СообщениеПользователю;
        Сообщение.Текст = "Не удалось загрузить файл на FTP-сервер, описание ошибки: " +
        Сообщение.Сообщить();

    КонецПопытки;

КонецПроцедуры
```

Задача

Выгрузка цен из 1С на FTP для обновления цен на сайте

Нужно организовать выгрузку информации о ценах из 1С на FTP-сервер.
Цены должны быть выгружены в файл формата ***.csv** следующей структуры:

Создадим форму с командами заполнить и загрузить:

```
&НаКлиенте
Процедура Заполнить(Команда)
    ЗаполнитьНаСервере();
КонецПроцедуры
```

```
&НаСервере
Процедура ЗаполнитьНаСервере()

    Объект.Цены.Очистить();

    Запрос = Новый Запрос;
    Запрос.Текст =
        "ВЫБРАТЬ
        |   ЦеныТоваровСрезПоследних.Товар.Наименование КАК Наименование,
        |   ЦеныТоваровСрезПоследних.Товар.Артикул КАК Артикул,
        |   ЦеныТоваровСрезПоследних.Товар.Штрихкод КАК Штрихкод,
        |   ЦеныТоваровСрезПоследних.Цена КАК Цена
        |ИЗ
        |   РегистрСведений.ЦеныТоваров.СрезПоследних(, ВидЦен = &ВидЦен) КАК ЦеныТоваровСрезПоследних";

    Запрос.УстановитьПараметр("ВидЦен", Объект.ВидЦен);

    РезультатЗапроса = Запрос.Выполнить();

    Выборка = РезультатЗапроса.Выбрать();

    Пока Выборка.Следующий() Цикл

        СтрокаТЧ = Объект.Цены.Добавить();
        ЗаполнитьЗначенияСвойств(СтрокаТЧ, Выборка);

    КонецЦикла;

КонецПроцедуры
```

```
&НаКлиенте
Процедура ЗагрузитьНаFTP(Команда)
    ЗагрузитьНаFTPНаСервере();
КонецПроцедуры
```

```
&НаСервере
Процедура ЗагрузитьНаFTPНаСервере()

    ПутьКВременномуФайлуНаСервере = ПолучитьИмяВременногоФайла("csv");

    Текст = Новый ТекстовыйДокумент;

    Текст.ДобавитьСтроку("name;article;barcode;price");

    Для каждого СтрокаТЧ Из Объект.Цены Цикл
        ТекСтрока = СтрокаТЧ.Наименование + ";" + СтрокаТЧ.Артикул + ";" + СтрокаТЧ.Штрихкод + ";" + Формат(СтрокаТЧ.Цена, "ЧДЦ=2; Ч
        Текст.ДобавитьСтроку(ТекСтрока);
    КонецЦикла;

    Текст.Записать(ПутьКВременномуФайлуНаСервере, КодировкаТекста.UTF8);
//загрузка файла на FTP-сервер
FTPСоединение = Новый FTPСоединение("195.69.187.77", , "user26101", "qNHzsRYBrF32", , Истина, 20);

Попытка

    FTPСоединение.УстановитьТекущийКаталог("/");
    FTPСоединение.Записать(ПутьКВременномуФайлуНаСервере, "/prices.csv");

    Сообщение = Новый СообщениеПользователю;
    Сообщение.Текст = "Файл успешно загружен на FTP-сервер";
    Сообщение.Сообщить();

Исключение

    Сообщение = Новый СообщениеПользователю;
    Сообщение.Текст = "Не удалось загрузить файл на FTP-сервер, описание ошибки: " + ОписаниеОшибки();
    Сообщение.Сообщить();

КонецПопытки;
```